

Quantum Integrated High-Performance Computing: Foundations, Architectural Elements and Future Directions

Suman Raj^{a,d,b,1}, Siva Sai^{c,d}, Yogesh Simmhan^b, Kyle Chard^a, Rajkumar Buyya^d

^a*Department of Computer Science, University of Chicago, 5730 S Ellis Ave, Chicago, IL, USA*

^b*Department of Computational and Data Sciences, Indian Institute of Science, Bengaluru, KA, India*

^c*Department of Electrical and Computer Engineering, National University of Singapore, Singapore*

^d*Quantum Cloud Computing and Distributed Systems (qCLOUDS) Lab, School of Computing and Information Systems, The University of Melbourne, Carlton, Melbourne, VIC, Australia*

Abstract

High-performance computing (HPC) has evolved over decades through multiple architectural transitions, from vector supercomputers to massively parallel CPU clusters and GPU-accelerated systems, continuously expanding the frontier of scientific discovery. With the emergence of quantum processing units (QPUs) as practical computational accelerators, a new opportunity arises to further extend this trajectory by integrating quantum and classical computing paradigms. This paper presents Quantum Integrated High-Performance Computing (QHPC), a visionary architectural framework that unifies CPUs, GPUs, FPGAs, and QPUs as first-class heterogeneous resources. We propose a layered system design comprising unified resource management, quantum-aware scheduling, hybrid workflow orchestration, middleware and programming abstraction, interconnect technologies, and a tiered execution model enabling seamless workload partitioning across classical and quantum backends. A central aspect of our vision is a strong user requests abstraction layer that exposes heterogeneous resources through a unified job submission interface, similar in spirit to existing schedulers such as Slurm, allowing users to describe workloads in a consistent template independent of underlying compute type or location. Drawing insights from prior accelerator integration eras, we outline how QHPC can support emerging workloads in quantum chemistry, materials discovery, combinatorial optimization, and climate modeling. We conclude by highlighting open challenges in building scalable, reliable, and programmable quantum-classical infrastructures that seamlessly connect global users to heterogeneous compute resources for future quantum-classical HPC ecosystems.

Keywords: Quantum Computing, High-Performance Computing, Classical-Quantum Interfaces, Foundations, Co-Scheduling

1. Introduction

High-Performance Computing (HPC) has been the principal engine of scientific computing for over five decades, enabling predictive modeling and large-scale data analytics across materials discovery [1], genomics [2], astrophysics [3], and many other domains. The trajectory of scientific computing has always been defined by its most fundamental unit of progress: the *architectural paradigm shift*. From the vector processors of the mid-1970s to the massively parallel processors of the 1990s, from the commodity cluster era of the early 2000s to the Graphics Processing Unit

Email addresses: sumanraj@uchicago.edu (Suman Raj), siva.sai@nus.edu.sg (Siva Sai), simmhan@iisc.ac.in (Yogesh Simmhan), chard@uchicago.edu (Kyle Chard), rbuyya@unimelb.edu.au (Rajkumar Buyya)

¹Suman Raj was a visiting student at d with primary affiliation at b, when the work in this paper started. Her current affiliation is a. Siva Sai was also a visiting student at d, and his current affiliation is c.

Correspondence to: School of Computing and Information Systems, The University of Melbourne, Carlton, VIC 3052, Australia.

(GPU) and Field-Programmable Gate Array (FPGA) accelerated heterogeneous architectures, each transition has been motivated by a clear gap between the computational demands of science and the capacity of the prevailing hardware model [4].

Contemporary exascale platforms such as Frontier [5], Aurora [6], and El Capitan exemplify this paradigm shift in architecture. Yet, even at exascale, irreducibly hard problem classes, such as molecular simulations of biologically and chemically relevant systems at full quantum-mechanical fidelity, remain computationally inaccessible [7]. It is precisely this wall of classical intractability that motivates integrating quantum computation into the HPC stack. We are now at a point in time where Quantum Processing Units (QPUs) are poised to become first-class computational accelerators within High-Performance Computing (HPC) ecosystems alongside GPUs, FPGAs, and Application-Specific Integrated Circuits (ASICs). The resulting paradigm, which we term “*Quantum Integrated High-Performance Computing*” (QHPC), represents not merely the addition of another heterogeneous accelerator, but a qualitative expansion of the computational state space available to scientific workloads: where GPU nodes scale the numerical throughput of classical algorithms linearly with node count, multi-QPU configurations could exploit entanglement-mediated quantum parallelism to extend the accessible Hilbert space exponentially with qubit count [8].

The heterogeneous accelerator model that is common today did not emerge trivially. The integration of GPUs into HPC required rethinking programming models, memory hierarchy co-design, high-bandwidth interconnects, compiler toolchains, and performance portability abstractions. Similarly, FPGAs have been integrated as reconfigurable accelerators for streaming, low-latency, and bit-level parallel workloads. The unifying lesson is that architectural heterogeneity is sustainable only when (a) the accelerator delivers a demonstrable computational advantage for well-defined workload classes, and (b) the system software stack can seamlessly orchestrate heterogeneous execution. In this context, QPUs represent the next logical accelerator in this trajectory, not as a wholesale replacement for classical HPC, but as a tightly integrated co-processor targeting problem classes with provable or empirically observed quantum advantage. Quantum computing derives its potential by exploiting physical principles to achieve exponential or polynomial speedups over the best-known classical algorithms for specific problem classes. Starting from Shor’s polynomial-time algorithm for integer factorization [9], Grover’s search algorithm for unstructured databases [10], and more recently, the Variational Quantum Eigensolver (VQE) [11], and Quantum Approximate Optimization Algorithm (QAOA) [12]: each targeting near-term hybrid classical-quantum execution and defines an algorithmic landscape for which QPU acceleration within HPC is theoretically well-motivated.

Major institutional initiatives have reinforced this trajectory of QHPC. The U.S. Department of Energy’s Oak Ridge National Laboratory (ORNL) is emerging as a flagship quantum-HPC integration facility, with a US\$ 125 million budget through 2030. ORNL is deploying an NVIDIA GB200 NVL72 system alongside IQM and Quantum Brilliance QPUs, creating the world’s first testbed for side-by-side benchmarking of GPU-accelerated simulation, QPU execution, and hybrid quantum-classical workflows at leadership-computing scale [13]. Japan’s RIKEN has demonstrated the largest-scale quantum-HPC co-execution to date: a co-located IBM Quantum Heron processor coupled to all 152,064 classical compute nodes of the Fugaku supercomputer in a closed-loop data exchange, enabling one of the largest quantum simulations of iron-sulfur molecular clusters [14]. The Center for Strategic and International Studies (CSIS) identified in March 2026 that integrating quantum computers into U.S. supercomputers is a strategic imperative, while noting that the U.S. currently lags Europe and Japan in the number of deployed hybrid quantum-supercomputing systems [15]. Europe has similarly committed that Germany and France now host Pasqal neutral-atom QPUs within their HPC facilities [16].

In this article, we propose a QHPC model that envisions a next generation hybrid computing architecture tightly coupling QPUs with conventional CPUs and GPUs under a unified workflow and resource management framework. The overarching goal is to treat the QPU as a *heterogeneous*

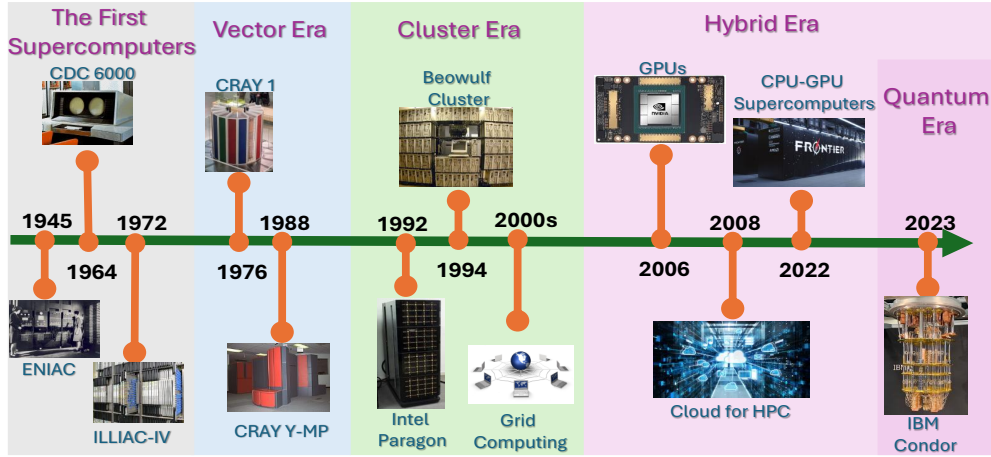


Figure 1: Timeline of the emergence of hybrid HPC and Quantum era

accelerator, analogous to GPUs in modern HPC, allowing quantum subroutines to be invoked seamlessly within large-scale classical workflows. This vision aims to bridge the gap between quantum and classical paradigms by providing an integrated execution model that maximizes hardware utilization, minimizes data transfer overhead, and abstracts complexity through intelligent scheduling and orchestration.

We make the following key contributions:

1. We briefly review the historical emergence of HPC, tracing the architectural evolution from vector processing through to the exascale GPU era, and motivate quantum integration with HPC (§ 2). A detailed review of the existing Quantum-HPC systems has also been added in § 6.
2. We offer a detailed review of the recent breakthroughs in quantum technologies along with fundamentals of quantum computing (§ 3). We further summarize the lessons learned from CPU-GPU co-design in HPC systems and map them onto the challenge of incorporating QPUs as first-class accelerators in HPC ecosystems (§ 4).
3. We propose a layered QHPC architecture that offers user interfaces for running classical-quantum workloads, and comprises a unified resource management system, quantum-aware scheduling, hybrid workflow orchestration, middleware, interconnect technologies for co-design, and workload partitioning across classical and quantum backends (§ 5).
4. We highlight a range of applications that stand to benefit from these integrated quantum-classical infrastructures (§ 7), and discuss key challenges that arise in such QHPC systems, many of which are likely to shape research directions over the next decade (§ 8).

Finally, we offer future research directions and our conclusions in §9.

2. Background

In this section, we briefly discuss the history and emergence of High-Performance Computing (HPC).

2.1. The First Supercomputers: 1940s – early 1970s

The origins of HPC trace back to the first digital computers, Electronic Numerical Integrator and Computer (ENIAC) developed in 1945 [17], which was originally designed to perform large-scale military and scientific computations necessitated by World War II. Almost two decades later,

in 1964, the CDC 6000 [18] by Control Data Corporation was launched as the first computer tagged as a “true supercomputer”. The CDC could process 500 KFLOPS up to 1 MFLOPS. The era also witnessed the development of ILLIAC-IV [19], the fourth Illinois Automatic Computer, designed as a parallel computer with a linear array of 256 64-bit processing elements and a goal of 1 GFLOP/s, but it could achieve only a fraction of this amount.

2.2. The Vector Supercomputing Era: 1975 – 1990

The processing speed in supercomputers was primarily achieved through vector processors and shared memory multiprocessing. Seymour Cray, the architect of the 6600 and other CDC computers, started his own company, Cray Research Inc., in 1972. The rise of vector processing led to deployment of Cray 1 [20] in 1976, which was a scalar and vector processor and capable of 133 MFLOPS. Subsequent developments led to systems such as Cray X-MP [21] in 1982, equipped with two vector processors, each capable of 200 MFLOPS and Cray Y-MP in 1988 equipped with upto 8 vector processors, each capable of 333 MFLOPS. There were other companies like Convex and Alliant that marketed vector supercomputers as well, during the 1980s.

2.3. The Cluster Era: 1990 – 2010

While vector processing worked well with 4 or 8 processors, memory contention prevented a 64 or 128-processor machine from working efficiently. This barrier required a transition from shared memory to distributed memory, where each processor has a dedicated memory space. The Message Passing Interface (MPI) standard [22, 23] (1994) was introduced as a unified programming model for distributed systems. The 1990s brought massively parallel processors (MPPs) like Intel’s Paragon (1992), Thinking Machines Corporation’s CM-1 (SIMD) to CM-5 (MIMD) and IBM’s SP2. These systems interconnected thousands of processors, enabling scalability for large problems and offering a processing speed of up to 59.7 GFLOPS. Further, the *Beowulf cluster movement* democratized HPC by enabling researchers to build clusters using commodity hardware and open-source software (typically Linux) [24]. *Grid computing* later emerged to link such clusters globally [25].

2.4. GPU and Hybrid Era: 2010 – Present

During the 2000s, the focus in high-performance computing (HPC) shifted from increasing individual processor clock speeds to scaling performance through *multi-core architectures*. This transition was soon complemented by the emergence of general-purpose *Graphics Processing Units (GPUs)*, which introduced unprecedented levels of parallelism and significantly enhanced energy efficiency. The introduction of programming frameworks such as CUDA [26] (2006) and OpenCL [27] (2009) enabled efficient utilization of these accelerators for scientific simulations, deep learning, and large-scale data analytics.

Modern HPC systems have now entered the *exascale era*, achieving sustained performance exceeding 1.38×10^{18} FLOPS. The *Frontier* supercomputer at Oak Ridge National Laboratory (ORNL) [5] was the first to surpass this milestone, marking a major leap in computational capability. More recently, *El Capitan* at Lawrence Livermore National Laboratory (LLNL) [28] overtook Frontier in 2024, reaching 1.7 exaFLOPS. Both systems exemplify *heterogeneous hybrid architectures*, where compute nodes integrate multiple multi-core CPUs with high performance GPUs. For instance, El Capitan uses a combined 11,039,616 CPU and GPU cores consisting of 43,808 AMD fourth Gen EPYC 24C “Genoa” 24-core 1.8 GHz CPUs (1,051,392 cores) and 43,808 AMD Instinct MI300A GPUs (9,988,224 compute units (228 each)).

Overall, the evolution of HPC over the years embodies the relentless pursuit of higher performance, scalability, and efficiency. As HPC approaches the limits of classical architectures, the focus is gradually shifting toward integrating quantum technologies as the next frontier in computation.

Table 1: Types of Qubits (as of October 2025)

Qubit Type	Typical Gate Speed	Medium of Information	Example System
Superconductivity	10 – 100ns	Josephson junctions and microwave resonators	IBM Eagle, Google Sycamore
Trapped-Ion	1 – 100 μ s	Hyperfine states of ions in electromagnetic traps	IonQ Aria, Honeywell H1
Photonics	<1 ns	Polarization or path of photons	Xanadu Aurora
Neutral-atom	1 – 100 μ s	Rydberg states of laser-cooled neutral atoms	QuEra Aquila, Atom Computing
Quantum Dots (Spin)	10 – 100ns	Electron spin in quantum dots or silicon structures	Intel Horse Ridge, TU Delft Spin Q
Topological	$\approx 1\mu$ s (<i>Theoretical</i>)	Majorana zero modes in superconducting nanowires	Microsoft Majorana 1
NV-Center	10 – 100ns	Electron spin in nitrogen-vacancy centers in diamond	IBM Quantum Diamond Lab
Fluxonium	100ns – 1 μ s	Superconducting loops with Josephson junction arrays	MIT Lincoln Lab Fluxonium Device
Silicon Donor	1 – 10 μ s	Spin states of phosphorus donors in silicon	UNSW SiMOS Qubits
NMR	10 – 100 μ s	Nuclear spin states in molecules under magnetic fields	Liquid-state NMR Systems at IBM & Oxford
Electron-on-Helium	1 – 10 μ s	Electron charge confined on the surface of helium	Experimental academic setups
CQED	10 – 100ns	Trapped atoms in high-finesse cavities	Yale QLab, ENS Paris

3. Embracing the Quantum Era

The advent of the quantum era signifies a paradigm shift in computational science, leveraging the principles of quantum mechanics to address classes of problems that are computationally infeasible for even the most advanced classical high-performance computing systems. By definition, “*A quantum computer is a computational device that exploits the principles of quantum mechanics, such as superposition, entanglement, decoherence and interference, to process information using quantum bits (qubits), enabling the solution of certain problems exponentially faster than classical computers.*” Analogous to Central Processing Unit (CPU) for classical computers, we have Quantum Processing Unit (QPU) as the brain of a quantum computer. It uses the behavior of particles like electrons or photons to make certain kinds of calculations much faster than processors in today’s computers [29].

Next, we discuss the fundamental mechanisms underlying quantum computation, beginning with the concept of qubits and the four core principles that govern quantum computing.

3.1. Qubits

A qubit (quantum bit) is the basic unit of quantum information, analogous to the classical bit. However, whereas the state of a bit can only be binary (either 0 or 1), the general state of a qubit according to quantum mechanics can be an arbitrary coherent superposition of all computable states simultaneously [30]. It can be represented as any proportion of 0 and 1 in the superposition of both states, with a certain probability of being a 0 and a certain probability of being a 1. We share details on types of qubits in Table. 1.

3.2. Principles of Quantum Computing

3.2.1. Superposition

A quantum state is commonly described using a wave function $|\psi\rangle$. The simplest single qubit wave function is a 2×1 column vector. In the two-dimensional plane, all states of the wave function of a single qubit can be formed by a linear superposition of two orthogonal basic vectors (referred to as base vectors), which are $|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. Mathematically, a qubit in superposition is represented as: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and β are complex numbers called probability amplitudes satisfying the normalization condition $|\alpha|^2 + |\beta|^2 = 1$. When we measure this qubit in the standard basis, the probability of outcome $|0\rangle$ with value “0” is α^2 and the probability of outcome $|1\rangle$ with value “1” is β^2 . Superposition enables quantum computers to evaluate multiple computational paths in parallel. For example, a system of n qubits can represent 2^n possible states simultaneously, providing an exponential increase in the effective state space compared to classical bits.

3.2.2. Entanglement

Entanglement is a unique quantum correlation between qubits, where the state of one qubit cannot be described independently of the state of another, even if they are spatially separated.

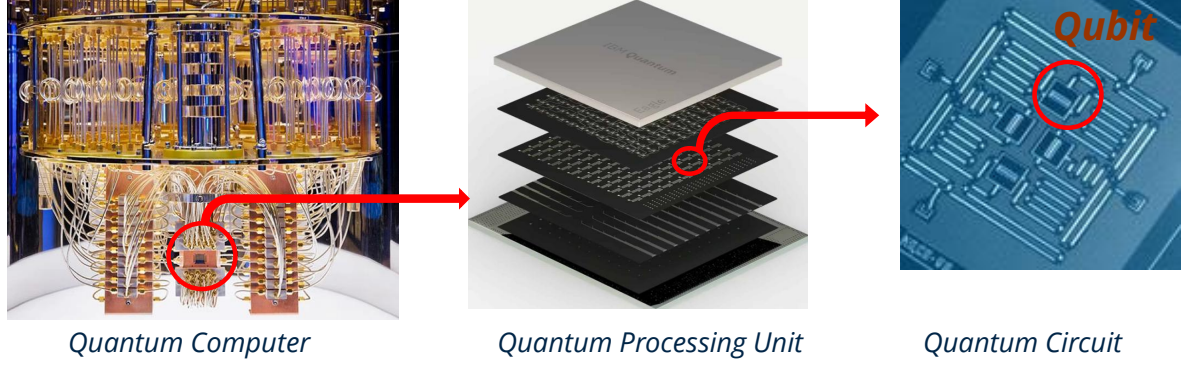


Figure 2: Representative anatomy of a Quantum Computer showcasing a quantum computer, a quantum processing unit, quantum circuit and a qubit. Picture Courtesy: IBM

When two or more quantum systems are entangled, their wavefunction cannot be expressed as a product of individual wavefunctions for each system. Instead, the systems are described by a single wavefunction that captures the correlation between them. The simplest illustration of entanglement is given by the *Bell states*. For example, two qubits in the entangled state, $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ form an *equal superposition* of the basis states $|00\rangle$ and $|11\rangle$, each occurring with a probability of $(1/\sqrt{2})^2 = 0.5$. Measuring one qubit instantaneously determines the state of the other, regardless of the distance separating them, exemplifying the non-classical and non-local nature of quantum entanglement. Entanglement enables non-classical correlations that are essential for quantum algorithms, teleportation, and error correction.

3.2.3. Decoherence

Decoherence refers to the process by which a quantum system loses its coherence and transitions from a quantum superposition to a classical state. This collapse of the quantum state can occur intentionally through measurement or unintentionally due to interactions with the external environment, such as thermal fluctuations or electromagnetic noise. When a qubit in superposition is measured, it collapses to one of its basis states, $|0\rangle$ or $|1\rangle$, with probabilities $|\alpha|^2$ and $|\beta|^2$, respectively. This phenomenon imposes a fundamental limitation on quantum computation, as the act of measurement inherently destroys superposition. Consequently, quantum algorithms must be meticulously designed to manipulate quantum amplitudes and interference prior to measurement, maximizing the likelihood of obtaining the correct computational outcome.

3.2.4. Interference

Quantum interference arises from the wave-like nature of quantum particles such as electrons or photons. When a particle is in a superposition of multiple states, these states can interfere with each other, often leading to constructive or destructive interference. During computation, multiple computational paths interfere with each other: correct paths are amplified (constructive interference) while incorrect paths cancel out (destructive interference). This mechanism underlies the efficiency of algorithms such as *Grover's search* [10], which amplifies correct solutions, and *Shor's algorithm* [9], which exploits interference to reveal periodic structures.

3.3. Quantum Processing Units (QPUs)

The first step to manufacture a QPU is to first realise the physical qubits using one of the technologies discussed in Table. 1. We show an illustration in Fig. 2 where superconducting qubits are realized as tiny electrical circuits made from superconducting materials such as aluminum or niobium, connected through *Josephson junctions*. These junctions act as non-linear inductors, enabling the circuit to exhibit quantum behavior, i.e., superpositions of current or voltage states. Once individual qubit circuits are designed, they are patterned onto silicon or sapphire substrates

Table 2: Emerging Quantum Processing Units (QPUs) as of October 2025

Organization	QPU	Launched In	Qubits	Qubit Technology
Caltech	Ocelot	Sept 2025	6,100	Neutral-Atom
Fujitsu, RIKEN	-	April 2025	256	Superconducting
D-Wave Quantum	Advantage 2	May 2025	~4,400	Quantum Annealing
IonQ	Tempo	March 2025	64	Trapped Ion
Microsoft	Majorana-1	Feb 2025	8	Topological Qubits
USTC	Zuchongzhi 3.0	Dec 2024	105	Superconducting
Google	Willow	Dec 2024	105	Superconducting
IBM	Heron (R2)	Nov 2024	156	Superconducting
CAS (China)	Xiaohong	Jan 2024	504	Superconducting
IBM	Condor	Dec 2023	1,121	Superconducting
Atom Computing	-	Oct 2023	1,180	Neutral-atoms
Intel	Tunnel Falls	June 2023	12	Spin
QuEra	Aquila	Nov 2022	256	Neutral Atom
Xanadu	Borealis	June 2022	216	Photonics

using nanofabrication techniques similar to those used in semiconductor manufacturing. Multiple qubits, along with resonators and coupling elements, are integrated onto a single *quantum processor unit* (or chip). This chip is then mounted inside a dilution refrigerator, which cools it to millikelvin temperatures so that the superconducting properties are maintained. The processor is connected via microwave lines, control electronics, and readout systems that generate and detect the qubits’ quantum states. Additional components like cryogenic amplifiers, shielding, and classical control hardware are included to stabilize and interface with the quantum processor. A list of the most popular and emerging QPUs have been shared in Table. 2.

3.4. State-of-the-Art Quantum Computing Architectures

State-of-the-art quantum computing architectures demonstrate these trends: (i) fault-tolerant error-correcting substrates (notably the surface-code family) and the hardware layouts that realize them, (ii) layered (modular) design patterns that separate physical control from logical layers to manage vastly different time-scales and responsibilities, and (iii) new low-overhead code families and modular network topologies that promise dramatic reductions in physical-qubit overhead. Representative examples include the silicon donor surface-code architecture that maps surface-code error correction onto a planar donor lattice with shared-control gate arrays [31], and the layered architecture framework that prescribes distinct physical, error-correction, logical, and application layers – enabling independent optimization of hardware and compilation stacks [32].

Recent advances move beyond surface codes such as advanced compilation-aware layout methods minimize surface-code resource overhead by exploiting long-range Bell-state preparation and edge-disjoint path routing [33]; modular “quantum-system-on-chip” (QSoC) hardware concepts show how tiled chips and entanglement multiplexing can scale physical qubit counts while simplifying control wiring and connectivity [34, 35]; bosonic mode encodings offer hardware-efficient logical qubits and new optical error-correction protocols that trade fewer modes for improved lifetime and loss resilience [36, 37]; and very recent LDPC-based, “bicycle” (bivariate bicycle) architectures propose modular high-rate quantum LDPC codes with explicit logical instruction sets and compilation strategies that can lower overhead by an order of magnitude relative to surface-code architectures for many workloads [38]. Together, these developments point to a hybrid engineering roadmap: optimize device-level layouts for native fault tolerance, adopt layered and modular designs for composability and control, and explore bosonic and qLDPC encodings to substantially reduce the physical-qubit cost of large-scale, fault-tolerant quantum computation.

4. Lessons Learnt from GPU and Hybrid Era

In this section, we discuss how the transition from traditional CPUs to GPU-accelerated and hybrid architectures has shaped contemporary approaches to parallelism, scalability, and programming models, offering valuable insights for the emerging quantum computing landscape.

4.1. Heterogeneity as a Design Principle

Modern HPC systems no longer rely on a single, uniform processor type; instead, heterogeneity has become a central design principle. Sophisticated designs now couple CPUs with GPUs, but extend further to domain-specific accelerators (TPUs, FPGAs, NPU, ASICs, etc.), unified memory architectures, and hierarchical scheduling for mixed workloads [39]. These resources are exposed through layered software stacks so applications can exploit each device’s strengths without rewriting algorithmic intent for every target. This principle is important for three major reasons: (1) *Cost-Effective Specialization*: Specialized units deliver much higher throughput and energy efficiency on their target kernels. (2) *Resource Multiplexing*: Different tasks in a workflow may best run on different devices, and (3) *Scalability*: Systems composed of heterogeneous elements can scale across a wider design space than homogeneous clusters.

For instance, El Capitan supercomputer uses AMD MI300A “APUs” that merge CPU and GPU chiplets and share a unified memory subsystem, enabling tighter CPU–GPU coupling and reduced data movement overhead [40]. Ongoing research demonstrates that heterogeneous CPU–GPU platforms dominate modern HPC, and effective performance depends heavily on coordinating compute and memory heterogeneity across tiers and devices [41]. Further studies show that even within a single GPU, heterogeneity (e.g., multi-instance GPUs, virtualized partitions) and task scheduling across heterogeneous processing elements can yield up to $1.87\times$ throughput gains [42].

This architectural evolution toward fine-grained and hierarchical heterogeneity in classical HPC systems offers valuable insights for emerging quantum–classical platforms, where managing the interplay between QPUs and conventional accelerators will be essential to achieve scalable hybrid computing.

4.2. Task Parallelism and Workload Decomposition

Task parallelism and workload decomposition are core engineering strategies for extracting performance from modern heterogeneous systems. At the core is the decomposition of a large computation into smaller, independently executable tasks (or kernels) and the mapping of those tasks onto a mix of processing resources (CPUs, GPUs, accelerators) to maximize utilization while respecting data dependencies and communication costs. However, the classical constraint laid out by *Amdahl’s law* remains relevant: the achievable speedup of any parallelization is bounded by the fraction of inherently serial work, so careful decomposition to minimize serial bottlenecks is a prerequisite for scalable performance.

GPUs are designed with thousands of small cores that can execute many tasks concurrently, making them ideal for computations that can be divided and run at the same time. The integration of mixed-precision Tensor Cores in modern GPUs, such as the NVIDIA A100, significantly enhances computational throughput for dense linear algebra operations when leveraged effectively by algorithms and compilers. These have a significant impact on how compute-bound kernels are expressed and scheduled, and improve performance by orders of magnitude for DNN workloads [43]. Task-based runtimes, such as StarPU let programmers express computation as a directed acyclic graph (DAG) of tasks with data dependencies [44]. These runtimes have been deployed in HPC environments, including linear algebra, dense and sparse solvers, and various data analytics pipelines, to achieve near-optimal task-to-resource mapping under varying load conditions. Further, runtimes like TimeGraph offer a real-time GPU scheduler at the device-driver level to provide prioritization and isolation of GPU workloads in multitasking environments, overcoming the limitations of non-preemptive GPU execution [45].

GPUs taught that massive *fine-grained parallelism* is the most scalable path to performance. However, achieving this required a deep transition from sequential CPU logic to *data-parallel* and *task-parallel* formulations. This lesson translates directly to quantum systems, where algorithmic parallelism (e.g., in superposition) must be explicitly harnessed through optimized circuit decompositions.

4.3. Software Ecosystem and Programmability

The past two decades have proven that hardware performance alone is insufficient and robust software ecosystems and programming models are equally critical. This is true for GPU accelerators, where CUDA and its companion libraries made GPUs accessible and productive, and is already shaping the trajectory of quantum computing (where SDKs such as Qiskit, Cirq and PennyLane form the bridge between algorithms and QPUs). CUDA created a full-stack developer path: a programming model (CUDA C/C++), highly optimized math libraries (cuBLAS, cuDNN), profilers and debuggers (Nsight, CUDA-GDB), and a distribution ecosystem (NVIDIA NGC containers) [46].

Unlike CUDA, which is proprietary to NVIDIA GPUs, OpenCL provides a cross-vendor, cross-platform programming model that allows developers to write kernels once and execute them on a variety of devices. OpenCL has been widely adopted in both research and industry to leverage heterogeneous clusters where different types of accelerators coexist. For instance, multi-vendor HPC systems, AI inference engines, and embedded devices often use OpenCL to exploit GPUs and FPGAs simultaneously without vendor lock-in. Modern ML frameworks, notably PyTorch [47] and TensorFlow [48], provide high-level libraries that simplify GPU programming by providing abstractions that manage the underlying parallel computations. This enables researchers to scale from a single GPU to multi-node, multi-GPU clusters with relatively modest engineering effort. Recent comparative studies and surveys emphasize that framework ergonomics, ecosystem libraries (optimizers, schedulers), and deployment tooling (containers, orchestration, model parallel runtimes) are the primary enablers of large-scale adoption [49].

This lesson has direct applications in quantum computing. Just like the software ecosystems unlocked the full potential of GPUs, quantum SDKs and frameworks are essential to harness the capabilities of QPUs and enable scalable quantum-classical applications.

4.4. Scalability through Abstraction and Modularity

HPC has long demonstrated that scalability requires both modular hardware design and software abstractions. The shift to networked clusters of modular compute nodes, each containing CPUs, GPUs, and other accelerators, enabled performance scaling through replication and hierarchical communication. For instance, the Modular Supercomputing Architecture (MSA) developed within the DEEP project series integrates different compute modules with specific performance characteristics, supporting a wide range of applications from computationally intensive simulations to data-intensive artificial intelligence workflows [50, 51]. According to recent studies, GPUs have become nearly synonymous with accelerators in the HPC space, comprising the overwhelming majority of accelerators in HPC clusters worldwide. Although originally designed for graphics rendering, GPUs have proven to be highly effective for parallel processing tasks, making them ideal for applications in AI, machine learning, and vast scientific simulations [52].

Software abstraction layers such as MPI (Message Passing Interface) for inter-node communication [23], OpenMP for intra-node parallelism [53], and container orchestration (e.g., Singularity [54], Docker [55]) allow applications to scale without requiring developers to manage low-level hardware details. Recent studies demonstrate that these software abstractions are essential for achieving both performance and scalability on current exascale and pre-exascale supercomputers. For example, performance analysis on the Frontier and El Capitan supercomputers shows that MPI combined with GPU-aware scheduling can improve throughput and

reduce communication overhead in large-scale scientific simulations [56]. Furthermore, NVIDIA introduced Multi-Instance GPU (MIG), which enables a single physical GPU to be partitioned into multiple isolated instances, each with dedicated compute cores and memory, enabling concurrent workloads without interference. Deploying many MIG instances across nodes improves job throughput and resource utilization, making it an essential mechanism for heterogeneous HPC clusters and AI workloads [57].

This experience with modular, heterogeneous CPU–GPU systems and software abstractions provides a roadmap for building scalable, hybrid quantum–classical platforms,

4.5. Co-Design of Hardware and Algorithms

The GPU and hybrid era reinforced the importance of hardware–software co-design for performance portability. Libraries and compilers were co-developed with hardware (e.g., cuBLAS, cuDNN), a principle which is now underpinning quantum algorithm design, such as matching circuit depth, qubit connectivity, and gate sequences, to specific QPU topologies.

Large-scale machine-learning systems illustrate this co-design in practice. Megatron-LM [58, 59] showed that careful intra-layer model parallelism combined with optimized communication patterns sustains high scaling efficiency across hundreds of GPUs when training multi-billion parameter transformers. Complementary frameworks, such as Microsoft DeepSpeed [60] and ZeRO [61], demonstrate that memory-centric decompositions (partitioning optimizer state, gradients, and parameters across devices) enable training trillion-parameter models by alleviating per-device memory pressure and reducing redundant communication. Architectures like the NVIDIA A100/H100, with exposed Tensor Cores and CUDA-X libraries, amplify these benefits, showing that performance and energy efficiency are fully realized only when algorithms and compilers are co-designed with hardware features.

This lesson of co-design carries directly into quantum computing, where tailoring algorithms to qubit topology, connectivity, and error characteristics is critical to achieving near-term performance.

5. QHPC System Architecture

In this section, we propose the QHPC architecture that constitutes a full-stack, heterogeneous computing architecture that treats QPU as a first-class accelerator within, not alongside, the classical HPC software and hardware stack. Our overarching expectation is that a user, from any institution or country, can submit a workload through a single, technology-agnostic entry point, and the QHPC system will automatically decompose, route, schedule, and execute that workload across the most appropriate combination of CPUs, GPUs, FPGAs, specialized AI accelerators, and QPUs available at that time. Figure 4 illustrates the QHPC architecture across five architectural layers: the User and Request Layer, the Workflow Management Layer, the Resource Management Layer, the Middleware and Abstraction Layer, and the Physical Compute Layer.

5.1. Design Philosophy: The Spectrum Between Slurm and Kubernetes

Before detailing the individual layers, a more fundamental architectural question must be addressed: what *execution model* appropriately captures hybrid quantum–HPC workloads? Classical HPC has largely converged on a batch-processing paradigm, using cluster managers and job schedulers like Slurm, where jobs are queued, resources are allocated exclusively, and execution is deterministic, time-bounded, and aware of system topology. In contrast, cloud-native systems adopt a microservice and container-orchestration model (e.g., Kubernetes), where long-lived services expose APIs, resources are elastically provisioned, and workloads are packaged for portability and resilience.

Hybrid quantum–classical workflows do not fit cleanly into either paradigm. Algorithms such as VQE and QAOA are inherently iterative and stochastic, coupling quantum circuit

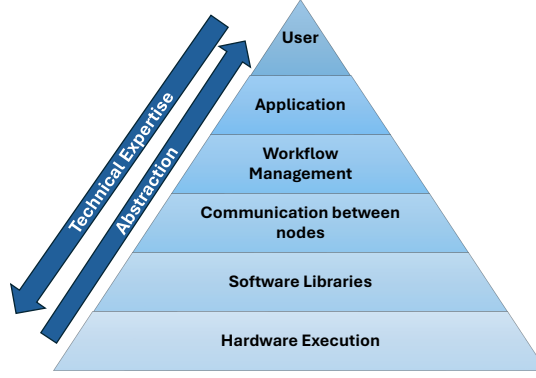


Figure 3: Abstraction vs Expertise for QHPC Systems

execution with classical optimization loops whose convergence behavior depends on measurement noise and shot statistics. At the same time, they require strict, topology-aware allocation of QPU resources to ensure reproducibility and isolation, which is incompatible with fully elastic service models. Consequently, hybrid workloads occupy an intermediate regime that challenges both batch-oriented determinism and service-oriented elasticity, motivating the need for a new execution abstraction tailored specifically to QHPC systems. This also anticipates the direction in which major vendors (IBM, NVIDIA, HPE) are converging their quantum-HPC integration blueprints.

5.2. Layer 1: User and Request Abstraction

The entry point to the QHPC system is a *unified submission interface* that is agnostic to the underlying compute modality. Users submit a Hybrid Workload Descriptor (HWD), a structured specification analogous to a Slurm or PBS job script but extended to include quantum circuit requirements, via one of three access channels. First, a command-line tool (`qhpc-submit`) compatible with `sbatch` that accepts HWDs in YAML or a domain-specific language and transparently forwards classical workloads to the HPC scheduler while routing quantum-enabled jobs through the QHPC workflow manager; second, a Python SDK exposing a `QHPCJob` abstraction for programmatic integration within Python, C++, or Fortran codes, unifying interactions with frameworks such as Qiskit primitives and OpenMP-Q-style quantum offloading; and third, a Kubernetes-fronted REST gateway secured via OAuth 2.0/OpenID Connect that enables global, allocation-free access for job submission, monitoring, telemetry, and result retrieval. Fig. 3 shows that higher abstraction makes systems easier to use and requires less expert knowledge.

The HWD itself comprises two logical components. The *classical logic descriptor* specifies traditional HPC requirements, including CPU/GPU counts, memory footprint, accelerator tier, MPI topology, data I/O paths, and execution time constraints. The *quantum logic descriptor* defines the quantum workload, including circuit representation (OpenQASM 3.0, QIR, or CUDA-Q kernels), qubit count and connectivity constraints, shot budget or target statistical confidence, supported QPU modalities (superconducting, trapped-ion, neutral-atom, or best-available), and fallback execution policies such as GPU-based emulation via `cuQuantum` or tensor-network simulation.

5.3. Layer 2: QHPC Workflow Management System (WMS)

The QHPC Workflow Management System is the *control plane* of the entire architecture. Its responsibilities are decomposing hybrid workloads into executable task graphs, managing inter-task data dependencies, scheduling tasks to the appropriate compute tier, and monitoring execution progress.

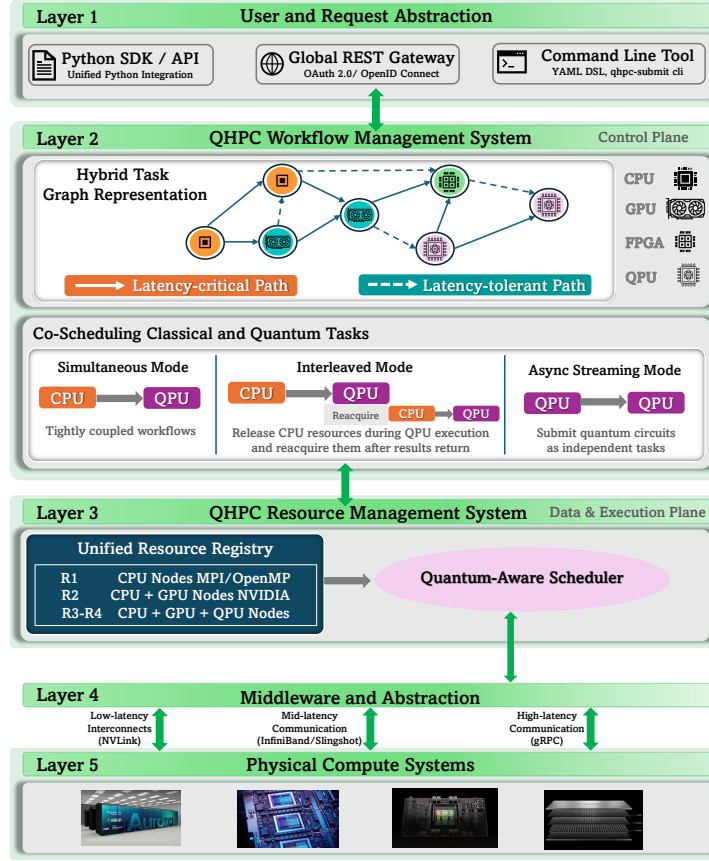


Figure 4: Quantum-Integrated HPC Architecture Layers

5.3.1. Hybrid Task Graph Representation

The WMS models a hybrid workload as a Directed Cyclic Task Graph (DCTG), extending classical DAG-based workflow model to support the iterative feedback loops inherent in variational algorithms, in which quantum measurement results are fed back into classical optimization nodes. Each node is typed as CPU (control flow and preprocessing), GPU (vectorized and ML workloads), QPU (quantum circuit execution producing probabilistic outputs on hardware or emulators), or FPGA (ultra-low-latency control tasks such as pulse generation and real-time QEC decoding that are unsuitable for GPU execution). The QHPC architecture is inherently extensible to new accelerator modalities, as any device exposed via a job scheduler plugin or Kubernetes Device Plugin can be incorporated into its unified resource hierarchy and scheduled natively.

5.3.2. Dependency Analysis and Critical Path Scheduling

Upon receiving a hybrid workload, the WMS performs *semantic dependency analysis* that goes beyond the structural data dependencies captured in the DCTG. It identifies *latency-critical paths*, chains of $CPU \rightarrow QPU \rightarrow CPU$ dependencies in which the classical side is idle while awaiting QPU results, and *latency-tolerant paths*, sequences of independent QPU circuit evaluations that can be batched and submitted in parallel to exploit QPU multi-programming or multiple-QPU parallelism. For latency-critical paths, the WMS routes the workload to co-located QPU and CPU resources connected via a low-latency fabric, targeting sub-millisecond round-trip times. For latency-tolerant paths, the WMS can distribute circuit evaluations across multiple QPUs at different geographic sites.

5.3.3. Co-Scheduling Classical and Quantum Tasks

The co-scheduling problem of allocating classical HPC nodes and QPU access is the central scheduling challenge of QHPC. The QHPC WMS tackles the co-scheduling of classical HPC nodes and QPU access through three per-job modes. *Simultaneous mode* co-allocates CPUs and QPUs for the full job enabling tightly coupled workflows like VQE and QAOA where classical and quantum steps alternate frequently. *Interleaved mode* releases CPU resources during QPU execution and reacquires them after results return, reducing idle time but adding scheduling overhead; the system dynamically switches to simultaneous mode when quantum phases are too short to benefit from interleaving. *Asynchronous streaming mode* submits quantum circuits as independent tasks while the classical pipeline continues execution, making it suitable for loosely coupled workloads such as quantum machine learning and batched kernel evaluation.

5.4. Layer 3: QHPC Resource Management System (RMS)

The QHPC Resource Management System is the *data and execution plane*, managing the inventory, allocation, scheduling, and runtime monitoring of all physical compute resources.

5.4.1. Unified Resource Model

The RMS maintains a *Unified Resource Registry* (URR) that lists every available compute resource as an object, along with its type and capabilities. QPU resources are modeled as dynamic resources. Unlike GPUs, whose performance is largely fixed, a QPU’s effective capability depends on its current calibration state, which can drift frequently. The RMS periodically polls vendor calibration APIs (e.g., every 15 minutes) and updates the resource profile with metrics such as gate fidelity and coherence time. This allows the scheduler to make calibration-aware decisions, selecting the most reliable QPU available for a given job.

5.4.2. Multi-Tier Resource Hierarchy

The QHPC resource hierarchy is organized into four tiers. *R1* comprises CPU-only nodes, multi-socket systems (e.g., AMD EPYC Genoa) interconnected via high-speed InfiniBand – serving classical workloads, MPI-parallel simulations, and hosting classical optimizers using MPI+OpenMP. *R2* includes CPU+GPU accelerated nodes (e.g., NVIDIA Blackwell-based systems), supporting GPU-accelerated simulation, deep learning, molecular dynamics, and quantum circuit emulation via cuQuantum or CUDA-Q. *R3* represents tightly integrated CPU+GPU+QPU nodes, where QPUs are co-located with classical resources and connected via low-latency interconnects (e.g., NVLink), enabling sub-microsecond feedback. *R4* consists of remote or cloud-accessed QPUs (e.g., IBM, IonQ, Amazon Braket), exposed via REST/gRPC APIs with higher latency, and thus suited for loosely coupled workloads. Across all tiers, the system supports heterogeneous QPU modalities, allowing the scheduler to map workloads to the most suitable hardware.

5.4.3. The Quantum-Aware Scheduler

The QHPC scheduler is designed as a layered policy engine that can be implemented on top of existing resource managers (e.g., Slurm) through extensible plugins and interfaces. The *classical scheduling layer* provides standard capabilities such as fair-share policies, backfilling, and accelerator-aware allocation for CPUs and GPUs. The *QPU availability layer* augments this with calibration-aware selection, using a Quantum Suitability Score (QSS) that combines gate fidelity, connectivity compatibility, queue wait time, and access latency to choose the most appropriate QPU; jobs are issued virtual QPU device tokens encapsulating this context. A *co-scheduling optimizer* handles tightly coupled hybrid jobs by jointly allocating classical resources and QPUs within shared time and latency constraints, while a fallback policy ensures progress under resource scarcity by dynamically routing workloads to GPU-based emulation, queuing for QPU access, or notifying users of degraded execution modes.

5.5. Layer 4: Middleware and Abstraction Layer

The Middleware Layer is the software stack that sits between the WMS/RMS and the physical hardware, providing hardware-agnostic programming abstractions, circuit compilation and transpilation, error mitigation, and communication protocols.

5.5.1. Quantum Intermediate Representation and the Compilation Pipeline

All quantum circuits submitted to QHPC are first lowered to the Quantum Intermediate Representation (QIR), an LLVM-based, hardware-agnostic IR that enables portable optimization and backend-specific lowering, analogous to LLVM in classical HPC. The compilation pipeline then proceeds in four stages: (i) logical optimization, including gate cancellation, commutation, and depth reduction; (ii) device mapping and qubit routing, where logical qubits are mapped to hardware with noise-aware placement and SWAP insertion based on current calibration data; (iii) pulse-level optimization (for supported superconducting QPUs), generating device-specific control pulses to minimize execution time; and (iv) error mitigation selection, where strategies such as ZNE, PEC, or CDR are chosen based on circuit characteristics and the target QPU’s noise profile, or omitted for fault-tolerant systems.

5.5.2. The Classical–Quantum Communication Protocol

Communication between classical HPC nodes and QPU control systems is handled through a *three-layer Quantum Communication Protocol Stack*. The *transport layer* uses low-latency interconnects such as NVLink, PCIe Gen 5, or CXL for co-located (R3) systems, achieving round-trip latencies of $< 4, \mu s$ (controller–GPU) and $< 10, \mu s$ (QPU–HPC), while remote (R4) QPUs are accessed via HTTPS/gRPC over Ethernet or InfiniBand. The *session layer* maintains persistent connections, abstracts vendor-specific APIs, and handles authentication, job submission, and device lifecycle events. The *data serialization layer* encodes shot-based outputs into efficient formats (e.g., NumPy arrays) and transfers them via a lightweight protocol (QRTP), optimized for high-throughput communication of 10^4 – 10^6 samples per circuit evaluation.

5.5.3. Quantum-Aware Programming Interface

The middleware exposes a *Quantum-Aware Programming Interface* (QAPI) with three abstraction levels. At the highest level, users access prebuilt hybrid algorithms (VQE, QAOA, QSVM, QAE) that take domain objects such as Hamiltonians or datasets and internally handle circuit generation, compilation, QPU execution, error mitigation, and post-processing, targeted at domain scientists without quantum programming expertise. The circuit level supports direct submission via OpenQASM 3.0, CUDA-Q, Qiskit, or PennyLane for users requiring fine-grained control over quantum workflows. At the lowest level, pulse APIs provide direct hardware control for R3 systems with pulse access, enabling calibration and device-level experimentation.

5.6. Layer 5: Physical Compute Layer

The physical layer of QHPC consists of compute hardware and a three-tier interconnect fabric. *Interconnect hierarchy*. Intra-node NVQLink provides sub-4 μs latency and ~ 64 Gb/s bandwidth for tight QPU–GPU coupling and real-time QEC, while inter-node InfiniBand NDR or Slingshot-11 supports MPI-scale communication. A wide-area quantum-federation overlay enables secure access to remote QPUs (R4) over WAN (10–100 ms latency), suitable only for latency-tolerant workloads. Superconducting QPUs require ~ 15 mK operation with 5–30 kW cryogenic overhead and strict isolation (magnetic shielding, vibration control), leading to physically separated QPU pods connected via FPGA-based control electronics to the HPC fabric. In contrast, photonic and NV-center QPUs can be rack-integrated with standard CPU/GPU systems.

Overall, the proposed QHPC architecture is designed to evolve gracefully: as QPU qubit counts increase, error rates decrease, and coherence times lengthen along published vendor roadmaps

(IBM targeting 10,000+ two-qubit gates by 2028; Google targeting fault-tolerant logical qubits by 2029), the R4-tier workloads of today, those currently requiring GPU-emulation fallback, will progressively migrate to R3 physical QPU execution, and the R3 tier will in turn host increasingly complex applications that today are only tractable classically.

6. Existing Quantum Integrated HPC Systems

It is essential to comprehend the current landscape of existing QHPC systems, both for integrating and improving upon them in future work. In this section, we discuss existing QHPC systems and their features.

Mantha et al. [62] introduced Pilot-Quantum, a QHPC middleware for task, resource, and workload management. Pilot-Quantum is built for quantum applications that are based on task parallelism, such as circuit cutting systems (used to divide and execute large quantum circuits) and hybrid algorithms (like variational approaches). Pilot-Quantum is built on the basis of a rigorous analysis of existing application execution patterns and quantum middleware systems. It also supports high-level programming frameworks such as PennyLane and Qiskit. The authors demonstrated the capabilities of the system through quantum machine learning, circuit cutting, and mini-apps (simple and representative kernels focused on performance bottlenecks) scenarios.

Shehata et al. [63] proposed a hardware-agnostic QHPC software to support both futuristic fault-tolerant quantum computers and the current noisy-scale quantum devices, thus presenting a comprehensive software stack. The proposed QHPC software includes standardized APIs for resource management, a quantum gateway interface, and a scheduling mechanism to handle both interleaved and parallel quantum-classical workloads. A few important elements of the framework include - a quantum programming interface to abstract hardware details; a quantum platform manager to integrate various quantum hardware systems; a resource management system to efficiently coordinate quantum and classical resources; and a tool chain for quantum circuit execution and optimization. Similar to Pilot-Quantum, the authors demonstrated the proposed framework with the implementation of hybrid quantum-classical algorithms like variational quantum linear solver, concluding the framework’s ability to handle complex workflows while considering resource utilization maximization.

Burgholzer et al. [64] introduce the Munich Quantum Software Stack (MQSS), a modular, multi-layered architecture for tightly coupled hybrid workflows that integrates QPUs as first-class HPC accelerators. Its core “middle-end” layer comprises a Quantum Resource Manager & Compiler Infrastructure (QRM & CI) built on a Multi-Level Intermediate Representation (MLIR), enabling hardware-aware, dynamic compilation and optimization driven by live system telemetry. This layer includes a bi-level scheduler that coordinates with classical HPC managers (e.g., Slurm) to orchestrate hybrid jobs and minimize idle time across classical and quantum resources. The ecosystem exposes high-level applications through front-end adapters and interfaces heterogeneous hardware via the Quantum Device Management Interface (QDMI), which abstracts device control and data acquisition protocols.

Guerreschi et al. [65] presented the Intel Quantum Simulator (IQS), a large-scale simulator capable of full state-vector simulation for quantum circuits up to 42 qubits by using HPC infrastructure. The architecture uses OpenMP to employ a hybrid parallelization strategy for shared memory and the Message Passing Interface (MPI) for distributed memory. At the MPI, the quantum states are partitioned across process nodes, and operations on local qubits are executed without the need for inter-process communications (the global qubits require data exchange between paired processes). The authors implement a “pool” functionality that divides global HPC resources into independent groups, which allows for parallel simulation of different quantum states. The proposed architecture enables cloud-ready deployment and accelerates specific workloads like stochastic noise modeling (implemented via ensemble averaging and noise gates) and variational algorithms like Quantum Approximate Optimization Algorithm

(QAOA) driven by classical Particle Swarm Optimization. IQS includes a Python wrapper that is implemented with the help of Pybind11, allowing it to work as an HPC backend for popular quantum software frameworks, including Cirq, Qiskit, and ProjectQ. It also includes Dockerfiles that can be built into Docker containers to facilitate easier deployment on HPC clusters and cloud computing platforms. IQS provides methods for direct computation of exact probabilities and expectation values from the state vector without collapsing the quantum state. This is in contrast to the physical quantum hardware that requires multiple shots to estimate outcomes. The proposed methodology was benchmarked on the SuperMUC-NG supercomputer.

With the goal of enabling large-scale simulations of quantum circuits with the help of HPC systems, Wang et al. [11] developed NWQ-Sim (Northwest Quantum Simulator), a high-performance quantum circuit simulator. The authors integrated NWQ-Sim with XACC, a programming framework for hybrid quantum-classical applications, and executed variational quantum eigensolver and quantum phase estimation for quantum chemistry problems at a large scale. NWQ-Sim includes particular optimizations to handle the large computational overhead of VQE simulations, such as caching and reusing post-ansatz states, gate fusion, and direct expectation value calculation. The authors demonstrate the potential of optimized simulators and HPC resources to advance quantum chemistry. For instance, a proposed component of the system - caching reduced the gates required for energy evaluation by 3 to 5 orders of magnitude.

Esposito et al. [66] proposed a strategy for the orchestration of hybrid quantum-classical workloads in QHPC systems. The proposed method interleaves classical and quantum tasks, making use of heterogeneous job launches. It operates under a multiple program multiple data (MPMD) paradigm to manage distinct quantum and classical tasks in parallel. The authors employ the message passing interface (MPI) for data exchange between components, and the Classiq software suite and Qiskit Aer simulator, respectively, for quantum circuit synthesis and quantum circuit simulation. As a part of their case study, the authors worked with the Harrow-Hassidim-Lloyd (HHL) quantum algorithm, which is used to solve linear systems of equations that are derived from the discretization of linear differential equations. They show that the extraction of state vectors and circuit synthesis are the most time-consuming processes and that communication overhead via MPI was negligible.

Schusler et al. [67] demonstrated the integration of quantum and HPC servers in a real-world setup. The authors integrated the Dutch national HPC Center (SURF) with the Quantum Inspire platform with the help of a distributed infrastructure consisting of two distinct Slurm clusters (C1 and C2). The HPC center acts as the main host (C1), which submits JSON-formatted batch jobs to the second cluster (C2) through a REST API. The second cluster is co-located with the QUantum Inspire API. The proposed system utilizes a fast task manager (dispatcher) to orchestrate interactions between the co-located classical runtime and the quantum runtime. A request/reply pattern is employed via ZeroMQ to minimize the latency to the microsecond scale. This allows the system to execute Python-based hybrid tasks iteratively, wherein the classical runtime continues to generate circuits for the quantum runtime until a stop condition is met.

Shehata et al. [68] proposed a quantum framework (named QFw) that is designed to streamline the “loose integration” of quantum simulation resources with on-premise HPC environments, and validated QFw on the Frontier supercomputer. This methodology improves over the traditional per-process simulator model and utilizes Slurm’s heterogeneous job feature to parallelly allocate distinct sets of compute nodes for the classical application, and a dynamic simulation environment for the quantum part. The proposed architecture orchestrates these resources with the help of a Distributed Execution Framework (DEFw) that employs Remote Procedure Calls (RPC) to manage communication between the simulation backends and the classical application. The workflow is managed by two important components - the quantum task manager (QTM) and quantum platform manager (QPM). The framework uses the PMIx runtime environment to execute the simulations to establish a distributed virtual machine. Chundury et al. [69] extended QFw by introducing a lightweight Python library to allow multiple front ends’ interaction with

QFw and by integrating it with Northwest Quantum Simulator (NWQ-Sim).

Caldwell et al. [70] proposed a framework, NVQLINK, designed to tightly couple HPC resources with a quantum system controller to support latency-critical workloads like quantum error correction. The core methodology used a scalable “real-time interconnect” built on commodity Ethernet and remote direct memory access (RDMA) technology. The authors defined four distinct time domains - physical, deterministic, application, and real-time to orchestrate this heterogeneous system and extended the Cuda-Q model with new abstractions, allowing the developers to use multi-level intermediate representation (MLIR) to compile unified C++ programs that address FPGAs, CPUs, and GPUs, within a single logic flow. The proposed framework implements distinct compilation workflows based on the latency sensitivity. The compiler enforces an ahead-of-time strategy for systems with high latency sensitivity, where the complete ISA programs are pre-compiled, executed, and uploaded to FPGAs. On the other hand, the framework supports just-in-time compilation and interactive execution or low-latency sensitivity systems. It achieved sub-4 microsecond round-trip latencies in a proof-of-concept setup.

Chen et al. [71] presented a middleware scheme designed for HPC to enable distributed quantum computing through a hybrid workflow. The proposed QHPC framework integrates classical distributed scheduling with multi-GPU acceleration to manage the computational intensity of quantum simulations. In the proposed framework, a variational quantum eigensolver is employed to approximate ground states efficiently. This is followed by a classical preprocessing stage where Convolutional Neural Networks are used for feature extraction on quantum data, which is subsequently re-encoded into a final quantum circuit layer for scalar classification output generation. The framework uses a middleware stack incorporating the NVIDIA cuQuantum SDK and PennyLane to address the issue of computational intensity of the simulations.

Zhan et al. [72] proposed a modular, hardware-agnostic framework that is designed to integrate quantum computing into existing HPC environments with three core extensions. Firstly, a quantum interface library connects traditional HPC languages (C, C++, Fortran) with Python quantum SDKs via a hybrid MPI client-server execution model. This separates quantum circuit synthesis and execution from classical pre-/post-processing, which allows for asynchronous parallelism managed by the Slurm workload manager, with the goal of minimizing QPU idle time. Secondly, the Adaptive Circuit Knitting (ACK) hypervisor works as a workload distributor that utilizes tensor network-based analysis for partitioning large quantum circuits into smaller ones, thus enabling execution on smaller-scale, distributed QPUs. Finally, the framework also incorporates a quantum compiler extension that ingests LLVM IR and Quantum IR (QIR) to generate hardware-retargetable binaries, with a goal of optimizing compile times and enabling front-end-agnostic execution.

Slysz et al. [73] present a hybrid classical-quantum HPC environment at the Poznań Supercomputing and Networking Center integrating two room-temperature ORCA Computing PT-1 photonic QPUs into a standard data-center infrastructure via Ethernet. These QPUs are coupled with NVIDIA V100 and H100 GPUs to support heterogeneous workloads, with an extended Slurm scheduler managing CPUs, GPUs, and QPUs in a multi-user setting. The software stack leverages NVIDIA CUDA-Q and exposes photonic QPUs as HTTP REST services, enabling execution of hybrid variational algorithms such as binary bosonic solvers and quantum neural networks. These workflows iteratively offload quantum subroutines while performing classical post-processing on HPC nodes. Elsharqawy et al. [74] proposed a unified quantum platform to standardize the low-level integration of different quantum hardware modalities into HPC ecosystems using a co-designed hardware and software architecture. At the software level, the proposed methodology utilizes a unified runtime library that translates platform-agnostic quantum intermediate representation code into a unified quantum instruction set architecture. The translated instruction set architecture is capable of handling both quantum and classical control flow operations, and they are offloaded to a generalized quantum control processor (QCP)

via shared memory. The QCP microarchitecture extends previous designs to be technology-agnostic at the hardware layer with the help of modular hardware accelerators like units for sorting and atom detection, that are dynamically activated to support specific physical quantum platforms like neutral atoms or superconducting circuits.

7. Applications

In this section, we discuss various applications from multiple domains that can benefit from our proposed QHPC framework (§5). We survey application domains across which this framework delivers value, ranging from classical HPC workloads already transformed by GPU acceleration to emerging quantum-native and hybrid quantum-classical use cases that can exploit QPUs as they mature.

7.1. Computational Chemistry, Drug Discovery, Materials and Molecular Simulation

In computational chemistry, the case for QPU integration is strongest on theoretical grounds. Classical HPC has advanced molecular simulation through density functional theory (DFT), coupled-cluster methods, and molecular dynamics, yet all of these face an exponential barrier: exact full configuration interaction (FCI) scales combinatorially with system size, rendering strongly correlated molecules classically intractable [75]. QPU-accelerated algorithms such as VQE and QPE offer a natural alternative, encoding fermionic wavefunctions directly in Hilbert space and circumventing the exponential memory overhead of classical representations [76], [77]. Similarly, in materials science, the HPC community has pursued classical-scale simulations for decades using density functional theory codes such as VASP, Quantum ESPRESSO, and CP2K. Here again, the fundamental limitation is the same as in computational chemistry: DFT exhibits cubic scaling with system size, restricting exact treatment to systems of fewer than a few thousand atoms, while critical phenomena in strongly correlated materials involve electronic correlations that DFT fails to capture reliably [78].

The practical significance of QHPC capability for drug discovery is also substantial. For instance, large-scale screening tasks evaluating millions of candidate ligand conformations for a drug-protein binding calculation can be offloaded to GPU-accelerated classical nodes leveraging ML force fields and docking engines. However, a reduced subset requiring chemically accurate binding free energies is selectively escalated to QPU-accelerated VQE or sample-based quantum diagonalization, supported by HPC-driven error mitigation [79]. Li et al. [77] demonstrated such a hybrid quantum computing pipeline for real-world drug discovery, targeting high-fidelity Gibbs free energy estimation for prodrug activation.

The energy storage sector presents the most near-term commercial motivation. Hybrid quantum-classical simulations of lithium battery electrolytes, including VQE-based calculations of ground and excited states for salts, have been demonstrated as early proof-of-concept QPU workloads in the context of the IBM quantum-HPC co-design program [80]. The U.S. Department of Energy’s ARPA-E Quantum Computing for Computational Chemistry (QC3) program has specifically identified quantum simulation of energy materials as priority use cases for the hybrid HPC-QPU paradigm [81, 82].

7.2. Genomics, Life Sciences and Bioinformatics

Genomics and life sciences represent one of the fastest-growing HPC application domains, driven by the continued decline in sequencing costs and the resulting exponential growth in genomic data volumes. Genome sequencing analysis pipelines are routinely executed on HPC clusters and increasingly on GPU-accelerated nodes, using frameworks such as NVIDIA Parabricks. Within the QHPC framework, classical bioinformatics pipelines continue to be served efficiently on the CPU and GPU tiers, while QPU-based quantum machine learning opens new capabilities for several technically challenging sub-problems [83]. Quantum machine learning classifiers,

particularly quantum support vector machines (QSVMs) and quantum neural networks, have been demonstrated for genomic variant interpretation, multi-omics integration, and drug-target interaction prediction, tasks that are characterized by high-dimensional feature spaces and limited labeled training data, conditions that favor the quantum kernel advantage [84]. Quantum annealing and QAOA have been applied to protein folding optimization as a QUBO problem, and variational quantum algorithms have been used to predict RNA secondary structure folding energies with higher accuracy than classical dynamic programming approaches on small instances. FreeQuantum is a hybrid quantum-classical pipeline for biomolecular free energy calculations, which in QHPC could be dynamically partitioned between QPUs for quantum-accelerated steps and classical HPC for sampling and ML tasks [85].

7.3. *Quantum-Accelerated Artificial Intelligence and Machine Learning*

AI workloads constitute the fastest-growing share of HPC center allocations globally. The QHPC framework, whose GPU-accelerated classical tiers already serve these workloads efficiently, additionally opens the possibility of QPU-assisted acceleration for specific AI sub-problems where quantum methods offer a theoretical advantage. Quantum machine learning (QML) encompasses a family of algorithms in which quantum circuits replace or augment classical neural network components. AI methods improve quantum hardware calibration, circuit compilation, and error correction, while quantum hardware accelerates AI training by exploiting quantum kernel methods, quantum sampling, and quantum feature maps that can represent exponentially large feature spaces with polynomial quantum resources [84], [86, 87]. Reinforcement learning presents a particularly compelling hybrid use case: during training of a quantum-enhanced RL agent for robotic control, a quantum critic network executed on a QPU can process the high-dimensional state representations produced by the environment, while the classical actor network, which must execute in real time during inference, runs entirely on CPU nodes with GPU accelerators [88].

7.4. *Earth Science, Climate Modeling and Computational Fluid Dynamics*

The integration of QPU accelerators introduces new capabilities for several sub-problems in the Earth system sciences that are computationally intractable at the classical scale. Ho et al. provide a systematic review of quantum computing applications to climate resilience and sustainability challenges, identifying QML models for climate pattern recognition, quantum optimization for renewable energy dispatch, and quantum-enhanced hydrological modeling as near-term use cases [89]. Quantum optimization algorithms are well-suited to the combinatorial problems embedded in climate-related logistics, such as the optimal placement of renewable energy generation facilities (wind farms, solar installations) subject to grid topology, transmission constraints, and the optimization of carbon capture [89].

Computational fluid dynamics (CFD) is one of the most demanding HPC workloads, as resolving turbulent flows at engineering scale Reynolds numbers requires multi-scale simulations that consume millions of CPU-GPU core-hours on leadership systems. The Navier-Stokes equations governing atmospheric and oceanic dynamics are the core computational kernel of climate models, and recent theoretical analysis has demonstrated analytic bounds suggesting that quantum algorithms can achieve exponential speedup over classical CFD solvers for turbulent flow simulation, a result with direct implications for cloud and convection parameterization in climate models as well [90, 91]. Complementarily, Syamlal et al. implemented and benchmarked a variational quantum CFD solver on real QPU hardware, establishing the feasibility of solving CFD-governing PDEs on near-term quantum processors and outlining a path toward industrial-scale adoption [92]. A comprehensive review categorizes quantum-CFD approaches into circuit-based algorithms, quantum-inspired tensor networks, and hybrid variational solvers [93].

7.5. *Financial Modeling and Combinatorial Optimization*

The financial services industry is one of the most computationally intensive non-scientific domains, spending billions of dollars annually on compute resources for risk management,

derivative pricing, portfolio construction, and fraud detection [94, 95]. The most theoretically grounded near-term quantum advantage in finance is in Monte Carlo simulation, where quantum amplitude estimation (QAE) achieves quadratic reduction in sample complexity relative to classical Monte Carlo [96]. For computation of Value-at-Risk (VaR), Conditional Value-at-Risk (CVaR), and derivative pricing under correlated stochastic processes, this speedup translates to either higher accuracy at fixed computational cost or equivalent accuracy at substantially reduced runtime, both outcomes of direct commercial value. Within the QHPC framework, a financial analyst can submit a portfolio-optimization task specifying the asset count, constraints, and accuracy targets. The scheduler dynamically maps small instances to CPU solvers, medium-scale problems to GPU-accelerated heuristics or quantum-inspired methods, and large, highly correlated cases to QAOA or variational circuits on QPUs. This mirrors hybrid quantum-classical deployments in industry, such as the collaboration of JP Morgan Chase with IBM, where quantum amplitude estimation was used for European option pricing on real QPU hardware, illustrating a pathway toward practical financial quantum acceleration [77].

Combinatorial and discrete optimization problems, such as vehicle routing, supply chain scheduling, facility location, network flow, and inventory management, constitute the largest category of industrially relevant NP-hard problems. Ahmed et al. present a recent systematic review identifying 20 studies in 2024 alone on quantum applications in supply chain management, spanning dynamic routing, demand forecasting, and disruption-resilient scheduling via quantum annealing and QAOA [97]. Similarly, the QED-C’s 2024 use-case assessment highlights route optimization, multimodal scheduling, and emissions-aware logistics as high-impact quantum opportunities, emphasizing continuous route optimization to reduce fuel costs and improve profitability [98]. Complementing this, Zaman et al. document the use of QAOA and quantum-inspired evolutionary methods for carbon-neutral supply chains, linking quantum logistics optimization directly to sustainability objectives [99].

Additional domains, including cybersecurity, cryptography, astrophysics, high-energy physics, and nuclear simulations, are also potential candidates that can benefit significantly from the QHPC framework. However, a discussion of these areas is deferred to future work.

8. Open Challenges

8.1. Heterogeneous Facility Co-Location: Physical and Operational Co-Design

The open problem: *How can a shared facility be designed and operated to support CPUs and diverse accelerators (GPUs, FPGAs, ASICs, and QPUs) with differing and often conflicting infrastructure requirements?*

Quantum processors impose infrastructure requirements that differ fundamentally and often conflict with those of classical HPC systems. Superconducting QPUs require cryogenic operation at millikelvin temperatures, strict magnetic shielding, and vibration isolation, while other modalities (e.g., trapped-ion, neutral-atom, photonic) demand optical stability, vacuum environments, and electromagnetic control. In contrast, HPC facilities are optimized for high-density CPU/GPU deployments with significant power draw, liquid cooling, and mechanical activity, which can create interference that degrades quantum coherence. The lack of standardized specifications for integrating QPUs alongside classical accelerators further necessitates bespoke, facility-specific engineering, as highlighted by recent Open Compute Project efforts [100].

This challenge raises several research questions: (i) Can unified thermal management architectures be developed to support both cryogenic QPU environments and high-density accelerator nodes without cross-interference, and what are the implications for power efficiency and facility design? (ii) What electromagnetic isolation and zoning strategies are required to preserve QPU fidelity in environments with high-speed interconnects and multi-kilowatt compute nodes? and (iii) Can modular, rack-scale quantum infrastructure be standardized to enable scalable and

repeatable deployment within HPC facilities? Addressing these questions is critical to transitioning QPUs from bespoke experimental systems to first-class components of production HPC environments.

8.2. Quantum-Aware Job Scheduling in Multi-Site Environments

The open problem: *How should a scheduler allocate and co-schedule jobs across distributed CPUs and heterogeneous accelerators, while accounting for QPU variability and the tight latency coupling of hybrid quantum-classical workflows?*

Classical HPC schedulers such as Slurm assume relatively homogeneous resources, predictable runtimes, and stable availability. QPUs violate these assumptions: their effective capacity depends on calibration state (which drifts over hours and requires periodic recalibration), execution is inherently probabilistic due to shot-based estimation, and end-to-end latency includes opaque queue times in remote quantum services [62]. Moreover, hybrid algorithms such as VQE and QAOA introduce tight feedback loops between the classical and quantum stages, where inefficient coordination leads to idle time on both sides, reducing overall system utilization [101]. Current approaches, including Pilot-Quantum’s pilot abstraction [101], flexible resource release during quantum phases [102], and Slurm SPANK QPU plugins [103], address individual facets of this problem but do not provide a unified scheduling theory for multi-site, multi-modal quantum-HPC environments. The problem is further complicated when the QPUs are geographically distributed across national supercomputing centers (e.g., JSC in Germany, CEA-TGCC in France, ORNL in the USA) and the classical HPC workload spans multiple sites via WAN. In countries with distributed national computing infrastructure, such as India’s network of CDAC centers, or Japan’s distributed supercomputing network, no single site will possess all resources simultaneously, making cross-site co-scheduling a critical requirement [15].

This raises key research questions: (i) Can fidelity-aware, latency-constrained scheduling policies jointly optimize classical utilization and QPU access under noise and calibration drift? (ii) How should schedulers manage QPU availability zones and failover across heterogeneous quantum devices? and (iii) Can strategies such as reinforcement learning enable adaptive scheduling in the presence of non-stationary QPU behavior, and converge fast enough for practical deployment [104]?

8.3. Quantum-Classical Workflow Orchestration and the Circuit-Cutting Problem

The open problem: *How should a workflow system decompose and orchestrate hybrid quantum-classical applications across heterogeneous resources, while supporting low-latency feedback, mid-circuit operations, and circuit partitioning when workloads exceed available QPU capacity?*

Hybrid quantum-classical algorithms such as VQE, QAOA, and QSVM are inherently iterative, with bidirectional data dependencies between quantum circuit execution and classical optimization. This breaks the Directed Acyclic Graph (DAG) assumption underlying existing HPC workflow systems (e.g., Pegasus, Nextflow, Parsl), necessitating dynamically reconfigurable workflows [105]. The challenge is compounded by mid-circuit measurement and feedforward, where error correction and control logic require sub-microsecond classical response times, far below the latency supported by current workflow managers.

Additionally, circuit cutting and knitting, which is the technique of decomposing a circuit too large for a single QPU into subcircuits executable on smaller QPUs, with classical post-processing to reconstruct the full output distribution, introduces exponential classical overhead in the number of cuts [101]. Recent empirical profiling shows that workload splitting, using circuit cutting for VQE and data-parallel batching for QSVM, can materially improve throughput and reduce quantum resource footprint (e.g., QSVM up to 3x faster and 3x lower qubit-seconds), but it introduces non-trivial classical overheads (e.g., reconstruction/merge costs that can reach tens of minutes in VQE) and accuracy-mitigation trade-offs that must be surfaced to the runtime and user [106].

The optimal circuit partitioning problem (minimizing the number of cuts subject to QPU size constraints) is NP-hard in general, and heuristic approaches scale poorly. Designing workflow systems that natively support quantum circuit semantics, including feedback loops, low-latency control, and scalable circuit partitioning, remains an open challenge with no deployed solution at scale. Complementarily, choreography of quantum-classical workflows across HPC centers demonstrates that packaging these optimizations as reusable orchestration patterns (fan-out, cross-cloud partitioning and asynchronous/adaptive polling) can yield large system-level gains (e.g., up to 53% faster execution and 80% lower cost show across hybrid clouds), while coping with long and uncertain QPU queues [107].

Research questions posed here are: (i) Can the standard workflow DAG model be extended to a directed cyclic graph with bounded depth that captures the iterative feedback structure of variational quantum algorithms, and what scheduling theory is applicable to such cyclic graphs with probabilistic edge weights? (ii) What is the fundamental lower bound on circuit knitting overhead for structured application families such as molecular Hamiltonians and QAOA-based MaxCut, and can exploitable problem structure reduce the classical reconstruction cost from exponential to polynomial scaling? (iii) Can a just-in-time (JIT) quantum circuit compiler be developed that dynamically re-partitions circuits at runtime based on evolving QPU calibration data, analogous to JIT compilation in GPU systems, to optimize fidelity and execution efficiency?

8.4. A Unified Quantum-HPC Programming Model

The open problem: *How can the decades-long MPI/OpenMP/CUDA ecosystem be extended to support first-class, portable quantum offloading, enabling HPC developers in C++ or Fortran to invoke QPU kernels with the same abstraction used for GPU acceleration?*

The HPC programming model is organized as a layered stack: MPI for distributed memory communication, OpenMP for shared-memory parallelism, and CUDA/HIP/SYCL for accelerator offloading. This ecosystem has matured over three decades and is deeply integrated into scientific computing codes. In contrast, quantum software frameworks remain largely circuit-centric and Python-driven (e.g., Qiskit, Cirq, PennyLane, CUDA-Q), with no native interoperability with MPI communicators, Slurm schedulers, or C/Fortran data layouts [108]. This mismatch in programming models, memory representations, and execution semantics forms a fundamental barrier to practical QHPC integration [108]. Early efforts attempt to bridge this gap: OpenMP-Q [109] proposes a pragma-based extension to OpenMP that enables quantum task offloading using the familiar `#pragma omp target` paradigm, dispatching quantum circuits to QPU backends from within standard C++ HPC application code; CONQUIRE [110] provides an open-source co-execution framework achieving VQE runtime reductions through parallelised OpenMP quantum offloading; and the QIR Alliance’s LLVM-based quantum intermediate representation [111] provides a portable circuit IR that can in principle be targeted by any HPC compiler. However, these approaches remain incomplete: they do not fully address encoding MPI-distributed data into quantum states, integrating probabilistic measurement outcomes into classical workflows, or enabling performance-portable execution across heterogeneous QPU modalities.

Possible research questions are: (i) Can an MPI-QPU collective communication primitive, analogous to `MPI_Allreduce`, be defined to aggregate shot-based measurement statistics across distributed QPU executions? (ii) What compiler analyses and transformations are required to automatically identify quantum-amenable subcomputations in legacy C++/Fortran HPC codes and translate them into QIR-compliant circuits? (iii) Can a portable performance model, analogous to the Roofline model, be developed for such systems to predict optimal classical-quantum workload partitioning given heterogeneous hardware characteristics?

8.5. LLM-Assisted and Autonomous Quantum-HPC Code Generation and Workflow Synthesis

The open problem: *Can large language models (LLMs) be leveraged to automatically synthesize, optimize, and deploy quantum-classical application code, while jointly accounting for hardware*

availability and target accuracy constraints, thereby lowering the programming barrier for QHPC systems?

The primary bottleneck to democratizing access to QHPC is not hardware availability but software expertise. Developing correct and efficient hybrid quantum–classical applications requires expertise spanning domain science, classical HPC programming, quantum algorithms, circuit compilation, error mitigation, and scheduler interaction. This concentration of knowledge restricts meaningful usage of quantum-HPC systems to a narrow community of quantum computing specialists, directly opposing the universal-access vision outlined in § 5. While LLMs have shown promise in generating syntactically valid quantum circuits in frameworks such as Qiskit and Cirq, translating between quantum SDKs, and suggesting algorithmic refinements, their ability to produce correct, hardware-aware, and performance-optimized hybrid HPC code remains unproven [86].

Key open questions include: (i) Can a domain-specific fine-tuned LLM consistently generate hybrid quantum–classical programs that are both algorithmically correct and hardware-adapted, as measured by compilation success, circuit fidelity, and convergence behavior on standard benchmarks? (ii) What formal verification frameworks can certify the correctness of LLM-generated quantum circuits prior to execution, and what is the computational overhead of such verification at NISQ-scale circuit sizes? (iii) Can an autonomous QHPC agent combining LLM-based code synthesis, hardware-aware performance modeling, and closed-loop feedback from QPU execution continuously improve its code generation and scheduling policies without human supervision?

8.6. Fault Tolerance, Resilience, and Reproducibility in QHPC Systems

The open problem: *How should a QHPC system be architected to ensure high availability, reproducibility, and graceful degradation under QPU and classical subsystem failures, particularly for tightly coupled hybrid workloads?*

Classical HPC resilience relies on checkpoint/restart, fault-tolerant communication (e.g., MPI), and deterministic error detection mechanisms such as checksums. These approaches do not translate directly to QPU-based systems: quantum states cannot be copied due to the no-cloning theorem, mid-circuit checkpointing collapses the computation via measurement, and quantum outputs are inherently probabilistic, requiring repeated execution to estimate observables. As a result, QPU faults can manifest as silent degradations, for example, calibration drift may bias expectation values without triggering explicit failure signals, leading to incorrect optimization trajectories in hybrid workflows. In fault-tolerant regimes, the coupling becomes even tighter: quantum error correction requires real-time classical decoding within sub-microsecond latencies, and decoder failures can directly result in logical qubit loss, as demonstrated in recent real-time QEC experiments by IBM [112]. This creates a fundamentally new resilience regime in which classical HPC reliability and quantum hardware stability are co-dependent, and where failure in either subsystem can irreversibly compromise the computation.

Several open research questions arise: (i) Can a quantum job reproducibility protocol be defined that specifies the minimal set of artifacts required to reproduce a quantum–classical result, and what are the associated storage and computational overheads at scale? (ii) What is the theoretical lower bound on classical compute resources required for real-time QEC decoding in a fault-tolerant QPU with $\mathcal{N}$ logical qubits, and how does this requirement scale as $\mathcal{N}$ grows from tens to thousands of logical qubits? (iii) Can a hybrid checkpointing mechanism be designed that preserves the state of variational optimizations (parameters, gradients, and convergence history) to enable seamless recovery after QPU failures without invalidating previously consumed shot budgets?

Collectively, these challenges define the research agenda for Quantum Integrated High-Performance Computing over the next decade.

9. Conclusions and Future Directions

In this paper, we present Quantum Integrated High- Performance Computing, which offers a unified architectural vision that integrates CPUs, GPUs, and QPUs as first-class resources within a common stack. This enables seamless execution of hybrid workloads while preserving portability and scalability across heterogeneous systems. By extending established HPC paradigms into the quantum era, the framework lays the foundation for a unified compute continuum for future scientific discovery.

Future research directions span both systems and architectural frontiers. An important direction is the energy-efficient and sustainable operation of large-scale hybrid infrastructures, where quantum, classical, and accelerator-rich resources must be co-optimized under power and thermal constraints. Another promising direction is quantum emulation at scale, where GPU-based QPU simulation becomes a first-class HPC workload, enabling development, debugging, and benchmarking of quantum algorithms before hardware maturity. Extending the hardware abstraction layer, we envision deeper integration of FPGAs as real-time quantum system controllers, error-correction decoders operating within microsecond coherence windows, and low-latency hybrid accelerators for tightly coupled classical-quantum routines. Collectively, these directions point toward a future where heterogeneous compute resources operate as a unified, programmable, and globally distributed compute continuum.

Acknowledgements

The research visits of Suman Raj and Siva Sai to the University of Melbourne are funded by an Australian Research Council (ARC) Discovery Project. Additionally, Suman Raj was supported by Prime Minister’s Research Fellowship, Ministry of Education, India. Yogesh Simmhan was supported by a grant from the National Quantum Mission, DST, India.

References

- [1] C. Chen, D. T. Nguyen, S. J. Lee, N. A. Baker, A. S. Karakoti, L. Lauw, C. Owen, K. T. Mueller, B. A. Bilodeau, V. Murugesan, et al., Accelerating computational materials discovery with machine learning and cloud high-performance computing: from large-scale screening to experimental validation, *Journal of the American Chemical Society* (2024).
- [2] L. Wang, H. Wang, et al., Big data in genomics: overcoming challenges through high-performance computing, *Computational Molecular Biology* (2024).
- [3] S. Habib, A. Pope, H. Finkel, et al., Hacc: Simulating sky surveys on state-of-the-art supercomputing architectures, *New Astronomy* (2016).
- [4] J. Shalf, The future of computing beyond moore’s law, *Philosophical Transactions of the Royal Society A* (2020).
- [5] S. Atchley, C. Zimmer, J. Lange, et al., Frontier: Exploring exascale, in: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC, 2023.
- [6] R. Stevens, J. Ramprakash, P. Messina, M. Papka, K. Riley, Aurora: Argonne’s next-generation exascale supercomputer, Tech. rep., ANL (Argonne National Laboratory (ANL), Argonne, IL (United States)) (2019).
- [7] Y. Cao, J. Romero, J. P. Olson, et al., Quantum chemistry in the age of quantum computing, *Chemical reviews* (2019).

- [8] K. A. Britt, T. S. Humble, High-performance computing with quantum processing units, *J. Emerg. Technol. Comput. Syst.* (2017).
- [9] P. W. Shor, Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer, *SIAM review* (1999).
- [10] L. K. Grover, A fast quantum mechanical algorithm for database search, in: *ACM symposium on Theory of computing*, 1996.
- [11] M. Wang, F. Hua, C. Liu, et al., Enabling scalable vqe simulation on leading hpc systems, in: *SC'23 Workshops of The International Conf. on High Performance Computing, Network, Storage, and Analysis*, 2023.
- [12] B. Bach, J. Falla, I. Safro, Mlqaoa: Graph learning accelerated hybrid quantum-classical multilevel qaoa, in: *IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2024.
- [13] Oak Ridge National Laboratory, [ORNL, NVIDIA, HPE Advance Quantum Computing, AI and HPC for Science](#) (November 2025).
- [14] IBM, [IBM Releases a New Blueprint for Quantum-Centric Supercomputing](#) (March 2026).
- [15] Center for Strategic and International Studies, [Pioneering Quantum-Supercomputing Integration: U.S. Leadership in the Next Computing Era](#) (March 2026).
- [16] Pasqal, [Inside Pasqal's 2026 Vision on Quantum for Industry and Research](#) (January 2026).
- [17] H. H. Goldstine, A. Goldstine, The electronic numerical integrator and computer (eniac), in: *The origins of digital computers: selected papers*, Springer, 1946.
- [18] J. Swensen, Control data 6600, in: *Encyclopedia of Parallel Computing*, Springer US, 2011.
- [19] W. Bouknight, S. Denenberg, D. McIntyre, J. Randall, A. Sameh, D. Slotnick, The illiac iv system, *Proceedings of the IEEE* (1972).
- [20] R. M. Russell, The cray-1 computer system, *Communications of the ACM* (1978).
- [21] M. August, G. Brost, C. Hsiung, A. Schiffler, Cray x-mp: the birth of a supercomputer, *Computer* (1989).
- [22] M. Snir, *MPI—the Complete Reference: the MPI core*, Vol. 1, MIT press, 1998.
- [23] D. W. Walker, J. J. Dongarra, *Mpi: a standard message passing interface*, Supercomputer (1996).
- [24] T. L. Sterling, *Beowulf cluster computing with Linux*, MIT press, 2002.
- [25] I. Foster, C. Kesselman, *The Grid 2: Blueprint for a new computing infrastructure*, Elsevier, 2003.
- [26] J. Nickolls, I. Buck, M. Garland, K. Skadron, Scalable parallel programming with cuda, *ACM Queue* (2008).
- [27] A. Munshi, The opencl specification, in: *IEEE Hot Chips 21 Symposium (HCS)*, 2009.
- [28] Lawrence Livermore National Laboratory, [El Capitan: NNSA's First Exascale Machine](#) (2024).
- [29] R. Merritt, [What Is a QPU?](#) (Jul. 2022).

- [30] M. A. Nielsen, I. L. Chuang, Quantum computation and quantum information, Cambridge university press, 2010.
- [31] C. D. Hill, E. Peretz, S. J. Hile, et al., A surface code quantum computer in silicon, *Science Advances* (2015).
- [32] N. C. Jones, R. Van Meter, A. G. Fowler, et al., Layered architecture for quantum computing, *Physical Review X* (2012).
- [33] M. Beverland, V. Kliuchnikov, E. Schoute, Surface code compilation via edge-disjoint paths, *PRX Quantum* (2022).
- [34] L. Li, L. D. Santis, I. B. Harris, et al., Heterogeneous integration of spin-photon interfaces with a cmos platform, *Nature* (2024).
- [35] F. Riera-Sàbat, W. Dür, A modular entanglement-based quantum computer architecture, *New Journal of Physics* (2024).
- [36] V. V. Albert, Bosonic coding: introduction and use cases, *arXiv:2211.05714* (2022).
- [37] J. Hastrup, U. L. Andersen, All-optical cat-code quantum error correction, *Physical Review Research*, APS (2022).
- [38] T. J. Yoder, E. Schoute, P. Rall, et al., Tour de gross: A modular quantum computer based on bivariate bicycle codes, *arXiv:2506.03094* (2025).
- [39] S. Mittal, J. S. Vetter, A survey of cpu-gpu heterogeneous computing techniques, *ACM Comput. Surv.* (2015).
- [40] J. Wahlgren, G. Schieffer, R. Shi, et al., Dissecting cpu-gpu unified physical memory on amd mi300a apus, in: *Inter. Symp. on Workload Characterization (IISWC)*, IEEE, 2025.
- [41] L. Fusco, M. Khalilov, M. Chrapek, et al., Understanding data movement in tightly coupled heterogeneous systems: A case study with the grace hopper superchip, *arXiv 2408.11556* (2024).
- [42] U. Saroliya, E. Arima, D. Liu, M. Schulz, Hierarchical resource partitioning on modern gpus: A reinforcement learning approach, in: *International Conference on Cluster Computing (CLUSTER)*, 2023.
- [43] NVIDIA Corporation, [NVIDIA Ampere Architecture Whitepaper \(A100 Tensor Core GPU\)](#) (2020).
- [44] C. Augonnet, S. Thibault, R. Namyst, P.-A. Wacrenier, Starpu: A unified platform for task scheduling on heterogeneous multicore architectures, in: *International Euro-Par Conference on Parallel Processing*, 2009.
- [45] S. Kato, K. Lakshmanan, R. Rajkumar, Y. Ishikawa, et al., {TimeGraph}:{GPU} scheduling for {Real-Time}{Multi-Tasking} environments, in: *USENIX Annual Technical Conference (ATC)*, 2011.
- [46] J. D. Owens, D. Luebke, N. Govindaraju, et al., A survey of general-purpose computation on graphics hardware, in: *Computer graphics forum*, Wiley Online Library, 2007.
- [47] A. Paszke, S. Gross, F. Massa, et al., Pytorch: An imperative style, high-performance deep learning library, *NeurIPS* (2019).

- [48] M. Abadi, P. Barham, J. Chen, et al., Tensorflow: a system for large-scale machine learning, in: *USENIX Conference on Operating Systems Design and Implementation (OSDI)*, 2016.
- [49] G. Nguyen, S. Dlugolinsky, M. Bobák, et al., Machine learning and deep learning frameworks and libraries for large-scale data mining: a survey, *Artificial Intelligence Review* (2019).
- [50] M. Riedel, R. Sedona, C. Barakat, et al., Practice and experience in using parallel and scalable machine learning with heterogeneous modular supercomputing architectures, in: *IEEE IPDPS Workshops*, 2021.
- [51] E. Suarez, N. Eicker, T. Lippert, Modular supercomputing architecture: from idea to production, in: *Contemporary high performance computing*, CRC Press, 2019.
- [52] D. Reed, D. Gannon, J. Dongarra, Hpc forecast: Cloudy and uncertain, *Communications of the ACM* (2023).
- [53] E. Gabriel, G. E. Fagg, G. Bosilca, , et al., Open mpi: Goals, concept, and design of a next generation mpi implementation, in: *European Parallel Virtual Machine/Message Passing Interface Users' Group Meeting*, Springer, 2004.
- [54] G. M. Kurtzer, V. Sochat, M. W. Bauer, Singularity: Scientific containers for mobility of compute, *PloS one* (2017).
- [55] D. Merkel, Docker: lightweight linux containers for consistent development and deployment, *Linux J.* 2014 (239) (Mar. 2014).
- [56] B. Wilfong, A. Radhakrishnan, H. A. L. Berre, et al., Testing and benchmarking emerging supercomputers via the mfc flow solver, *arXiv:2509.13575* (2025).
- [57] B. Li, T. Patel, S. Samsi, V. Gadepally, D. Tiwari, Miso: exploiting multi-instance gpu capability on multi-tenant gpu clusters, in: *Proceedings of the 13th Symposium on Cloud Computing*, 2022.
- [58] D. Narayanan, M. Shoeybi, J. Casper, et al., Efficient large-scale language model training on gpu clusters using megatron-lm, in: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.
- [59] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, B. Catanzaro, Megatron-lm: Training multi-billion parameter language models using model parallelism, *arXiv 1909.08053* (2020).
- [60] J. Rasley, S. Rajbhandari, O. Ruwase, Y. He, Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters, in: *ACM SIGKDD international conference on knowledge discovery & data mining*, 2020.
- [61] S. Rajbhandari, J. Rasley, O. Ruwase, Y. He, Zero: Memory optimizations toward training trillion parameter models, in: *Inter. Conf. for High Performance Computing, Networking, Storage and Analysis*, IEEE, 2020.
- [62] P. Mantha, F. J. Kiwit, et al., Pilot-quantum: a middleware for quantum-hpc resource, workload and task management, in: *Inter. Symp. on Cluster, Cloud and Internet Computing (CCGrid)*, IEEE, 2025.
- [63] A. Shehata, P. Groszkowski, T. Naughton, et al., Bridging paradigms: Designing for hpc-quantum convergence, *Future Generation Computer Systems* (2025).

- [64] L. Burgholzer, J. Echavarria, P. Hopf, et al., The munich quantum software stack: Connecting end users, integrating diverse quantum technologies, accelerating hpc, in: Proceedings of the SC Asia and Inter. Conf. on High Performance Computing in Asia Pacific Region, 2026.
- [65] G. G. Guerreschi, J. Hogaboam, F. Baruffa, N. P. Sawaya, Intel quantum simulator: A cloud-ready high-performance simulator of quantum circuits, Quantum Science and Technology (2020).
- [66] A. Esposito, J. R. Jones, S. Cabaniols, D. Brayford, A hybrid classical-quantum hpc workload, in: International Conference on Quantum Computing and Engineering (QCE), 2023.
- [67] O. Schüsler, A. Torres-Knoop, J. Dijkshoorn, C. Hollemans, B. Van Der Vlies, R. Versluis, Towards a dutch hybrid quantum/hpc infrastructure, in: IEEE QCE, 2023.
- [68] A. Shehata, T. Naughton, I.-S. Suh, A framework for integrating quantum simulation and high performance computing, in: International Conference on Quantum Computing and Engineering (QCE), 2024.
- [69] S. Chundury, A. Shehata, T. Naughton III, et al., Qfw: A quantum framework for large-scale hpc ecosystems, Tech. rep., ORNL (2024).
- [70] S. A. Caldwell, M. Khazraee, E. Agostini, et al., Platform architecture for tight coupling of high-performance computing with quantum processors, arXiv preprint arXiv:2510.25213 (2025).
- [71] K.-C. Chen, X. Li, X. Xu, Y.-Y. Wang, C.-Y. Liu, Quantum-classical-quantum workflow in quantum-hpc middleware with gpu acceleration, in: Inter. Conf. on Quantum Communications, Networking, and Computing (QCNC), IEEE, 2024.
- [72] X. Zhan, K. G. Johnson, A. Esposito, et al., A full stack framework for high performance quantum-classical computing, arXiv preprint arXiv:2510.20128 (2025).
- [73] M. Slys, P. Rydlichowski, K. Kurowski, et al., Hybrid classical-quantum supercomputing: A demonstration of a multi-user, multi-qpu and multi-gpu environment, arXiv preprint arXiv:2508.16297 (2025).
- [74] A. Elsharkawy, X. Guo, M. Schulz, Integration of quantum accelerators into hpc: Toward a unified quantum platform, in: International Conference on Quantum Computing and Engineering (QCE), IEEE, 2024.
- [75] S. McArdle, S. Endo, A. Aspuru-Guzik, et al., Quantum computational chemistry, Reviews of Modern Physics (2020).
- [76] A. Peruzzo, J. McClean, P. Shadbolt, et al., A variational eigenvalue solver on a photonic quantum processor, Nature communications (2014).
- [77] W. Li, Z. Yin, X. Li, et al., A hybrid quantum computing pipeline for real world drug discovery, Scientific Reports (2024).
- [78] J. Kim, et al., Machine learning for accelerating energy materials discovery: Bridging quantum accuracy with computational efficiency, Advanced Energy Materials (2026).
- [79] R. Santagati, A. Aspuru-Guzik, R. Babbush, et al., Drug design on quantum computers, Nature Physics (2024).

- [80] M. A. G. A. Barroca, et al., Quantum simulations of battery electrolytes with VQE-qEOM and SQD: Active-space, arXiv:2509.13826 (2025).
- [81] ARPA-E, [U.S. Department of Energy Announces \\$30 Million to Use Quantum Computing for Groundbreaking Chemistry and Materials Science Simulations](#) (2024).
- [82] S. Kim, I.-S. Suh, T. S. Humble, et al., Harnessing quantum computing for energy materials: Opportunities and challenges, ACS Energy Letters (2026).
- [83] L. Wang, H. Wang, et al., Big data in genomics: overcoming challenges through high-performance computing, Computational Molecular Biology (2024).
- [84] R. M. Devadas, T. Sowmya, Quantum machine learning: A comprehensive review of integrating ai with quantum computing for computational advancements, MethodsX (2025).
- [85] J. Günther, T. Weymuth, M. Bensberg, et al., How to use quantum computers for biomolecular free energies, arXiv:2506.20587 (2025).
- [86] Y. Alexeev, M. H. Farag, T. L. Patti, et al., Artificial intelligence for quantum computing, Nature Communications (2025).
- [87] F. Rodríguez-Díaz, D. Gutiérrez-Avilés, A. Troncoso, et al., A survey of quantum machine learning: Foundations, algorithms, frameworks, data and applications, ACM Computing Surveys (2025).
- [88] G. Acampora, A. Ambainis, N. Ares, et al., Quantum computing and artificial intelligence: status and perspectives, arXiv:2505.23860 (2025).
- [89] K. T. M. Ho, K.-C. Chen, L. Lee, et al., Quantum computing for climate resilience and sustainability challenges, in: Inter. Conf. on quantum computing and engineering (QCE), IEEE, 2024.
- [90] X. Li, X. Yin, N. Wiebe, et al., Potential quantum advantage for simulation of fluid dynamics, Physical Review Research (2025).
- [91] R. D. Peddinti, S. Pisoni, A. Marini, et al., Quantum-inspired framework for computational fluid dynamics, Communications Physics (2024).
- [92] M. Syamlal, C. Copen, et al., Computational fluid dynamics on quantum computers, AIAA Aviation Forum and ASCEND (2024).
- [93] G. Malinverno, J. Blasco Alberto, A review of the current state-of-the-art of quantum computing for CFD: Approaches, advantages, and limitations, Aerotecnica Missili & Spazio (2025).
- [94] I. Dimoulkas, et al., From portfolio optimization to quantum blockchain and security: A systematic review of quantum computing in finance, Financial Innovation (2025).
- [95] J. Zhou, Quantum finance: Exploring the implications of quantum computing on financial models, Computational Economics (2026).
- [96] N. Stamatopoulos, D. J. Egger, Y. Sun, et al., Option pricing using quantum computers, Quantum (2020).
- [97] M. F. Ahmed, T. Nasir, et al., Quantum computing applications in supply chain information and optimization: Future scenarios and opportunities, Information (2025).

- [98] QED-C Use Cases Technical Advisory Committee, [Quantum Computing for Transportation and Logistics](#), Tech. rep., Quantum Economic Development Consortium (QED-C), SRI International (March 2024).
- [99] S. Zaman, S. Vudugula, S. K. Chebrolu, R. Saha, Ai and quantum computing for carbon-neutral supply chains: A systematic review of innovations, Available at SSRN 5209788 (2025).
- [100] Open Compute Project, [Data Center Integration of Quantum Information Infrastructure](#) (November 2025).
- [101] P. Mantha, F. J. Kiwit, N. Saurabh, S. Jha, A. Luckow, Hybrid quantum-HPC middleware systems for adaptive resource, workload and task management, arXiv:2604.03445 (2025).
- [102] R. Rocco, S. Rizzo, M. Barbieri, et al., Dynamic solutions for hybrid quantum-hpc resource allocation, in: QCE, IEEE, 2025.
- [103] U. Bacher, M. Birmingham, C. D. Carothers, et al., Quantum resources in resource management systems, arXiv 2506.10052 (2025).
- [104] H. Nguyen, M. Usman, R. Buyya, Qfor: a fidelity-aware orchestrator for quantum computing environments using deep reinforcement learning, ACM Transactions on Quantum Computing (2026).
- [105] S. S. Cranganore, V. De Maio, et al., Paving the way to hybrid quantum-classical scientific workflows, Future Generation Computer Systems (2024).
- [106] T. Khare, R. Majumdar, R. Sangle, A. Ray, P. V. Seshadri, Y. Simmhan, Parallelizing quantum-classical workloads: Profiling the impact of splitting techniques, in: 2023 IEEE International Conference on Quantum Computing and Engineering (QCE), 2023, pp. 990–1000. doi:10.1109/QCE57702.2023.00113.
- [107] V. Jha, S. Srivastava, T. H. Pal, V. M. Kavitha, R. Majumdar, T. Khare, P. V. Seshadri, V. Kulkarni, A. Ray, Y. Simmhan, Choreography and profiling of quantum-classical faas workflows on hybrid clouds, in: 2025 IEEE 25th International Symposium on Cluster, Cloud and Internet Computing (CCGrid), 2025, pp. 01–11. doi:10.1109/CCGRID64434.2025.00069.
- [108] C. Rallis, et al., Interfacing quantum computing systems with high-performance computing systems: An overview, arXiv:2509.06205 (2025).
- [109] S. Mittal, A. Mahesh, F. Mueller, Openmp-q: Quantum task offloading in openmp, in: International Workshop on OpenMP, Springer, 2025.
- [110] A. Mahesh, F. Mueller, Conquire: A co-execution environment for quantum and classical resources, in: QCE, IEEE, 2025.
- [111] QIR Alliance, [Quantum Intermediate Representation \(QIR\)](#) (2022).
- [112] IBM, [IBM Delivers New Quantum Processors, Software, and Algorithm Breakthroughs on Path to Advantage and Fault Tolerance](#) (Nov 2025).